

Scaling FastAPI Applications

What is Scaling

Scaling is the process of increasing your application's capacity to handle more users, more requests, and more data without degrading performance. There are two fundamental types of scaling.

Vertical Scaling (Scale Up):

Vertical scaling means increasing the resources of a single server. This includes adding more CPU cores, more RAM, faster storage, or better network connectivity. Vertical scaling is simple - you upgrade your server specification. However, it has limits. There is a maximum server size, and costs increase exponentially. A single point of failure also remains.

Single Server: Use Gunicorn with Uvicorn workers to utilize multiple CPU cores. This allows one server to handle many concurrent requests simultaneously.

Horizontal Scaling (Scale Out):

Horizontal scaling means adding more servers to share the load. Instead of one powerful server, you have many smaller servers working together. This is the preferred approach for production applications. It provides better fault tolerance, linear cost scaling, and no theoretical limit to how much you can scale.

Multiple Servers: Deploy your app in containers (Docker) and use an orchestrator like Kubernetes to run many replicas. A Load Balancer (like Nginx or AWS ALB) will then distribute traffic across these servers.

Cloud Instance

Cloud Instance is a virtual server that runs on someone else's infrastructure (like AWS, Google Cloud, Azure, DigitalOcean). You rent computing resources (CPU, RAM, storage) from a cloud provider and access them over the internet.

Cloud Deployment is the process of installing and running your application on cloud infrastructure. Your application runs on virtual servers managed by cloud providers. Focusing on coding not on machine capabilities

In easy word

Imagine you need a place to live.

- Cloud Instance = Renting an apartment
- You pay monthly rent
- Landlord handles maintenance, repairs, and utilities
- You can move to a bigger apartment when needed
- You do not own the building

Key Characteristics:

- Owned and managed by cloud provider (AWS, Google, Azure, etc.)
- Accessed over the internet
- Pay-as-you-go pricing (hourly or monthly)
- Can scale up or down easily
- Provider handles hardware maintenance
- Data stored in provider's data centers

Local Instance

Local Instance is a server that runs on your own physical hardware, either on your personal computer or in your own data center. You own and manage everything.

Local Deployment is the process of installing and running your application on your own physical servers. Your application runs on hardware you own and manage.

Simple Explanation:

Imagine you need a place to live.

- Local Instance = Owning a house
- You buy the property
- You handle all maintenance, repairs, and utilities
- Expanding requires buying more property
- You have complete control

Key Characteristics:

- Owned and managed by you or your company
- Accessed through local network or internet
- One-time hardware purchase cost
- Scaling requires buying new hardware
- You handle all maintenance and repairs
- Data stored on your own premises

Database Scaling at 70% Load

70% load means your database is using 70% of its available resources (CPU, memory, disk I/O, or connections). This is a warning threshold that indicates you should plan for scaling soon.

Options to scale

Option 1: Add Read Replicas

Read replicas are exact copies of your primary database that handle read operations. The primary database handles all write operations (INSERT, UPDATE, DELETE), while replicas handle read operations (SELECT)

Advantages:

- Easy to set up (most managed databases support this)
- Minimal application changes (route reads to replicas)
- Significant load reduction on primary
- High availability (if primary fails, replica can become primary)

Disadvantages:

- Does not help with write-heavy workloads
- Replicas may be slightly behind primary (replication lag)
- More complex queries that need latest data must go to primary:
- Take more disk space as same copy of db on one disk

Option 2: Upgrade Existing Server (Vertical Scaling)

Vertical scaling means increasing the resources of your existing database server (more CPU, more RAM, faster storage).

Before Vertical Scaling:

- ❖ Small truck (4 CPU, 16GB RAM)
- ❖ Can carry 100 packages per trip
- ❖ Truck is 70% full (70 packages)
- ❖ Need more capacity

After Vertical Scaling:

- ❖ Large truck (16 CPU, 64GB RAM)
- ❖ Can carry 400 packages per trip
- ❖ Same 70 packages now fill only 17.5% of truck
- ❖ Plenty of room for growth

Same truck route, bigger truck.

Advantages:

- No application changes required
- Quick to implement (cloud providers allow resizing)
- No data migration needed
- Simple to understand and manage

Disadvantages:

- More expensive (exponential cost increase)
- Has upper limit (maximum server size)
- Single point of failure still exists
- Downtime during upgrade (usually few minutes)

Option 3: Add New Sharded Database (Horizontal Scaling)

Database sharding means splitting your data across multiple database servers. Each server (shard) stores a portion of your data.

Advantages:

- Unlimited horizontal scaling (add more shards)
- Write capacity increases with more shards

- Each shard can be sized appropriately
- Failure of one shard does not affect others

Disadvantages:

- Very complex to implement
- Application must know about sharding
- Cross-shard queries are difficult
- Rebalancing shards is complex
- Requires significant development effort

PostgreSQL Query Optimization Techniques

PostgreSQL Query Pipeline:



Parser: Checks for valid SQL syntax.

Rewriter: Applies any "Rules" (e.g., View definitions) to transform the query.

Planner/Optimizer (Critical Stage): This is the "brain." It uses table statistics to evaluate thousands of possible execution paths (Join orders, Index vs. Seq scan) and chooses the one with the lowest "cost."

Planner does this in three steps:

- 1-generate all possible execution strategies.
- 2-estimate the cost of each strategy.
- 3-pick the one with the lowest estimated cost.

This is called cost-based optimization (CBO)

Executor: Follows the plan created by the Optimizer, physically reading blocks from disk or memory (Buffer Cache) and returning the rows